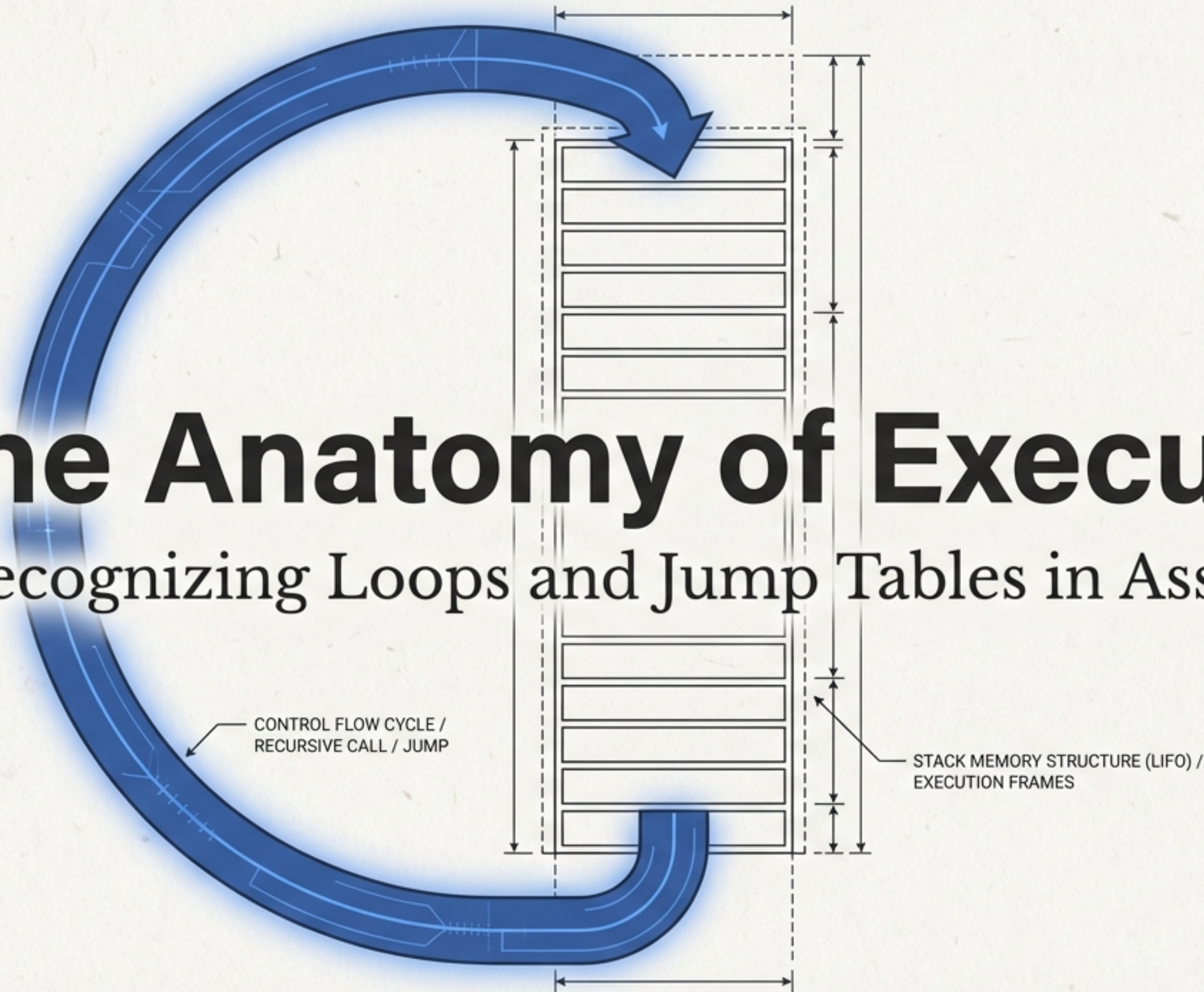


The Anatomy of Execution

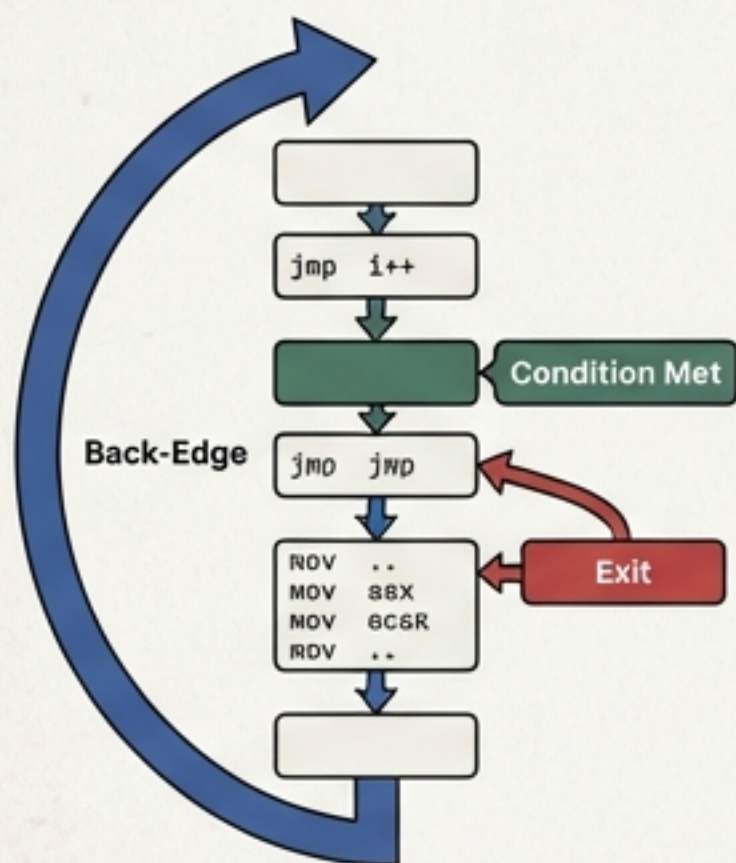
Recognizing Loops and Jump Tables in Assembly



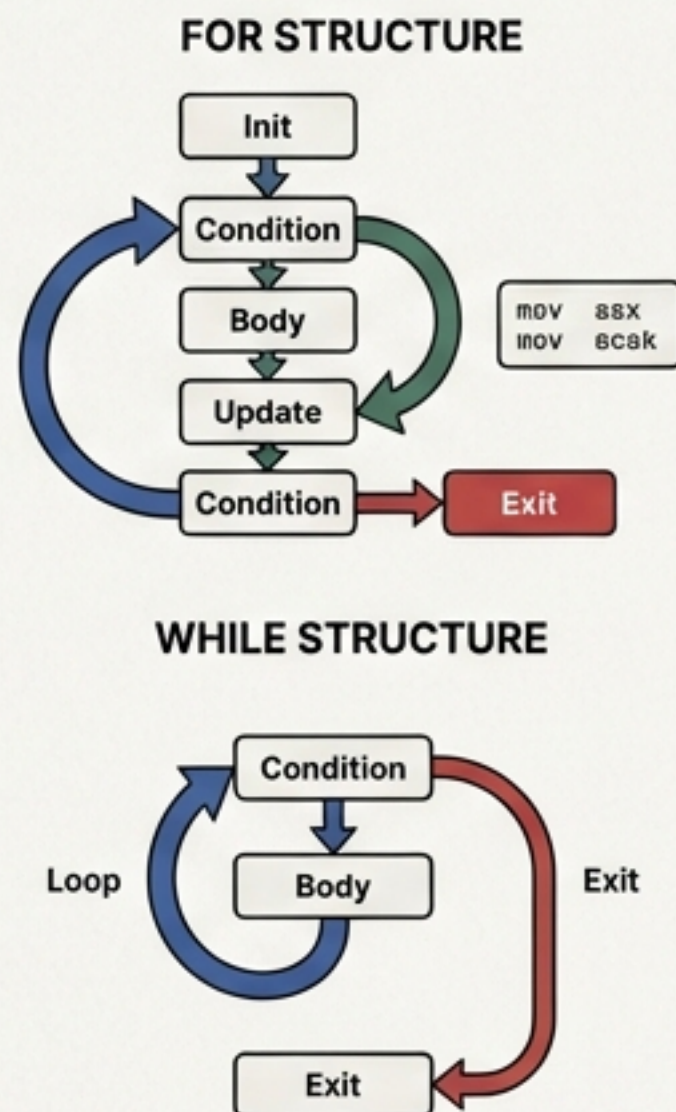
Logic is Circular, Assembly is Linear

Assembly code appears as a straight list of instructions, but the actual logic is full of cycles and forks.
To reverse engineer effectively, we must ignore the noise and identify the high-level shapes.

Visual Loops



Control Flow

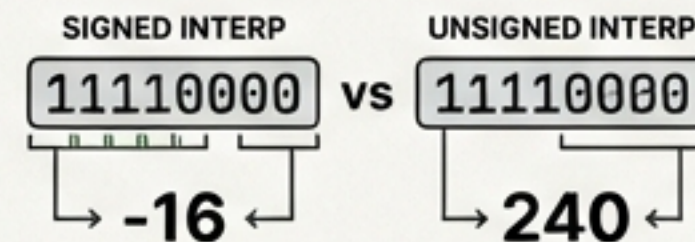


Data Forensics

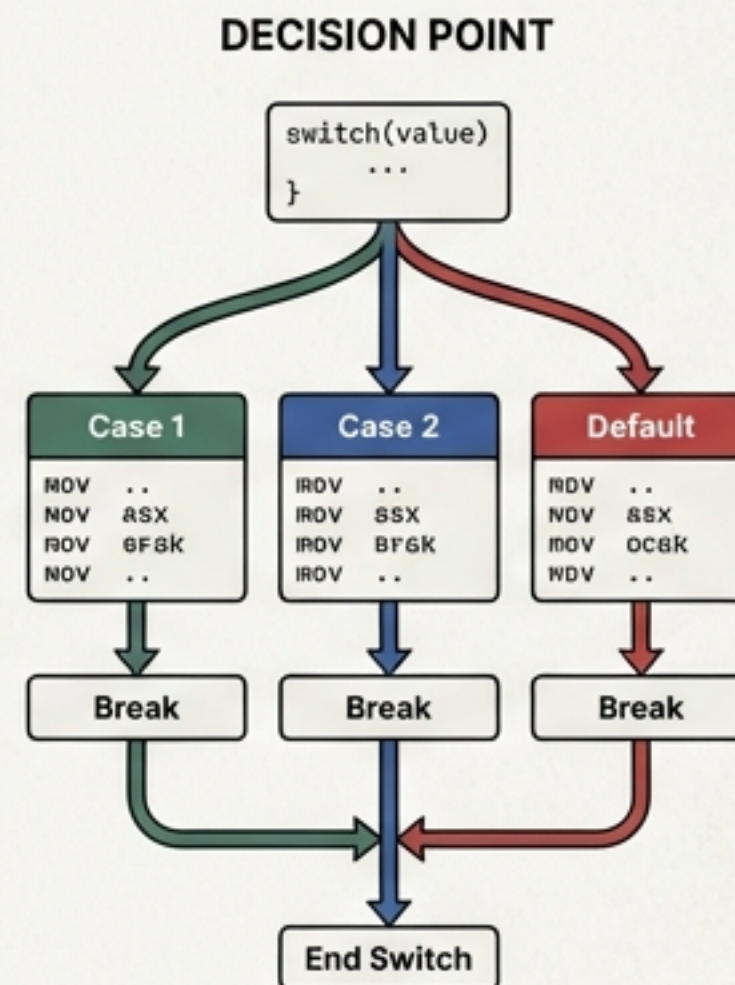


DATA INTERP

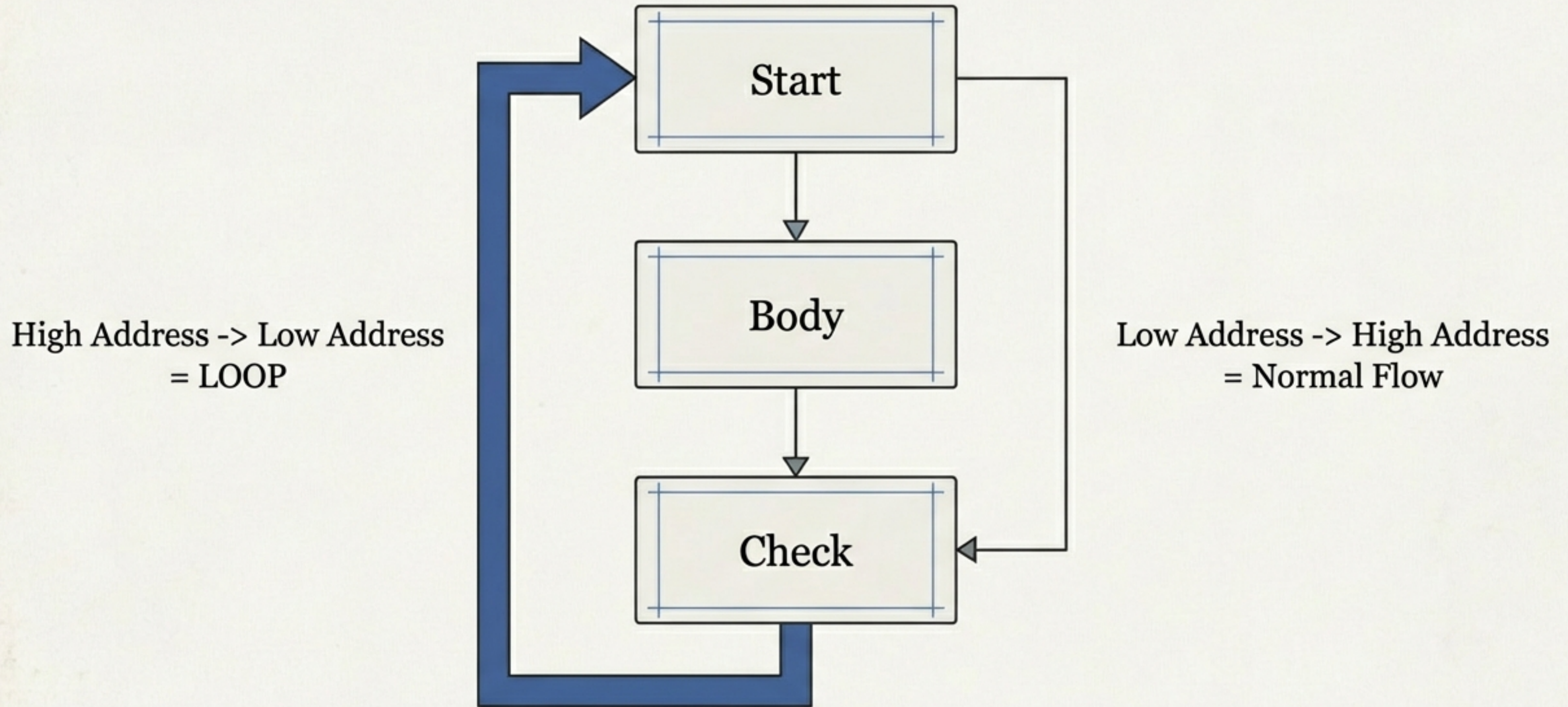
Signed vs Unsigned



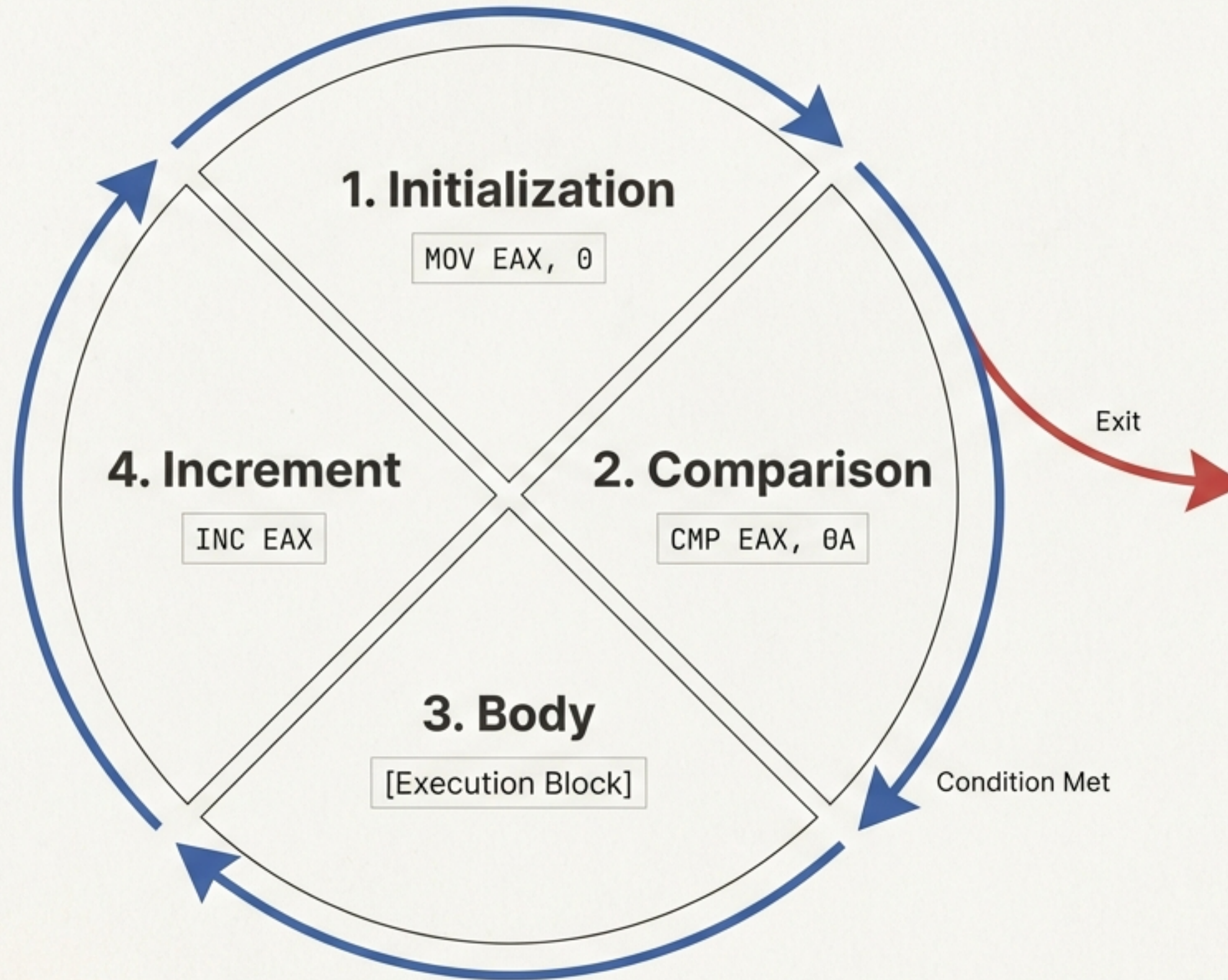
Master Decisions



The Back-Edge Indicator

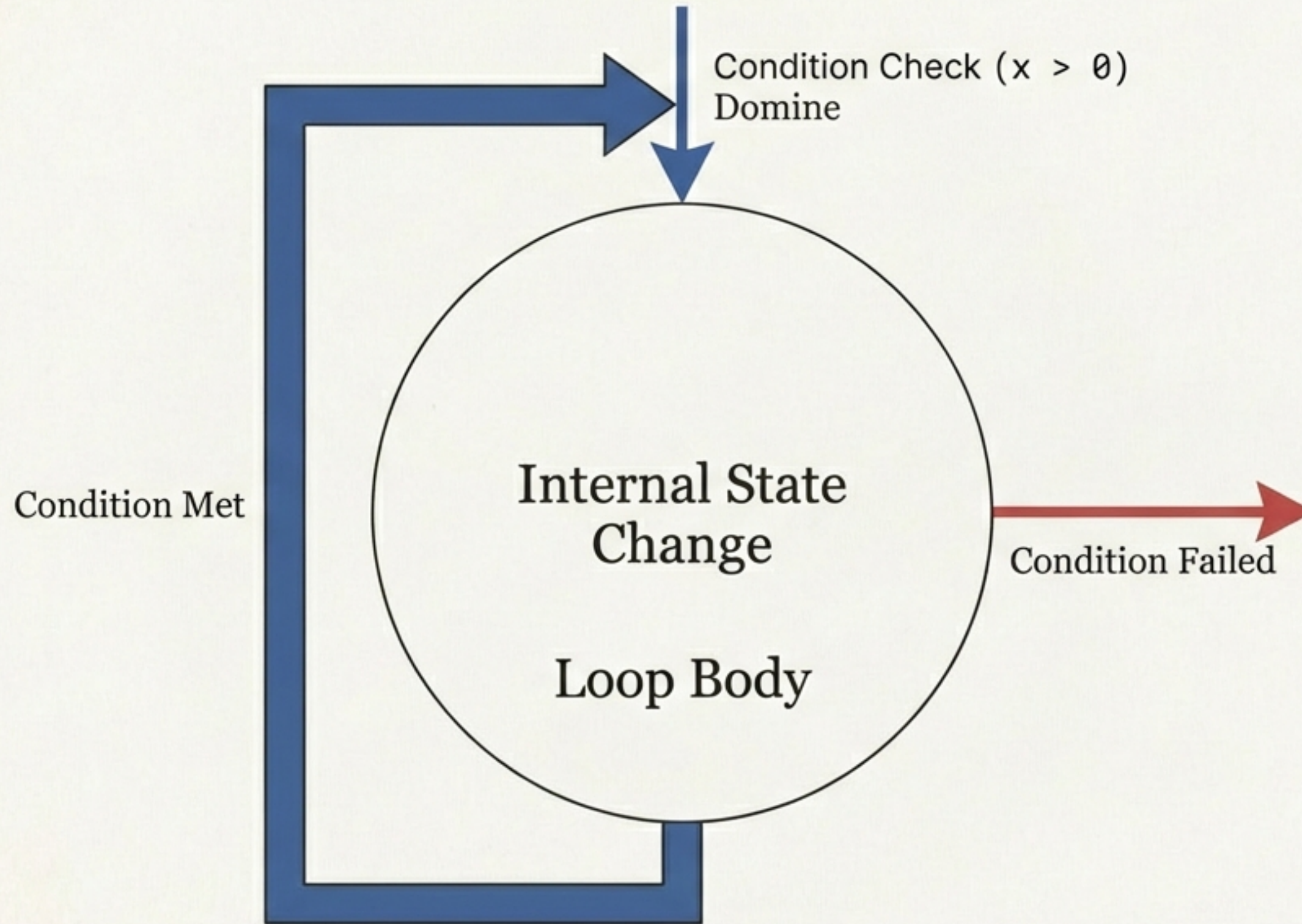


Deconstructing the "For" Loop



Identifying these four distinct pillars allows you to mentally reconstruct the original C code structure.

Variation: The "While" Loop

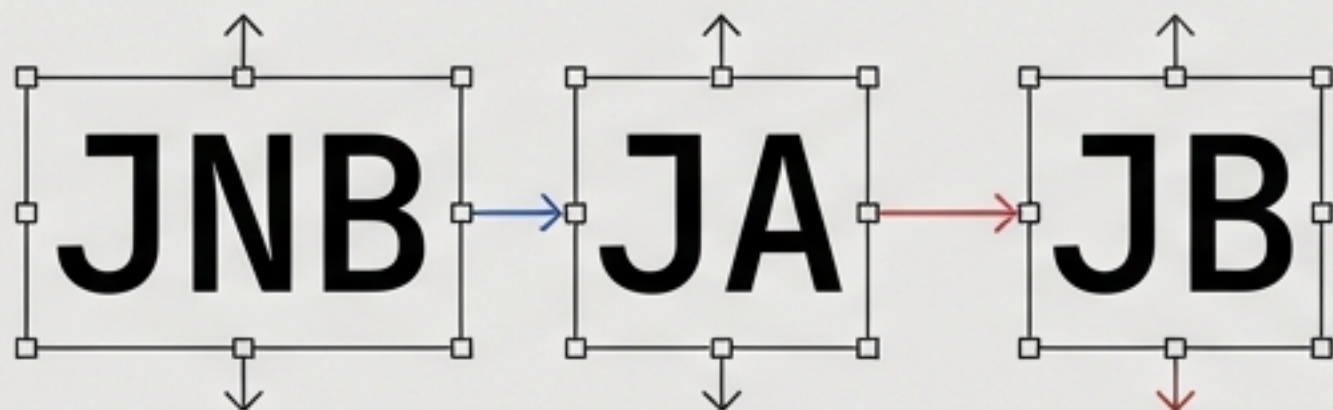


Unlike the "For" loop, the "While" loop has no external increment step. The state change happens naturally inside the body.

Forensics: Inferring Data Types

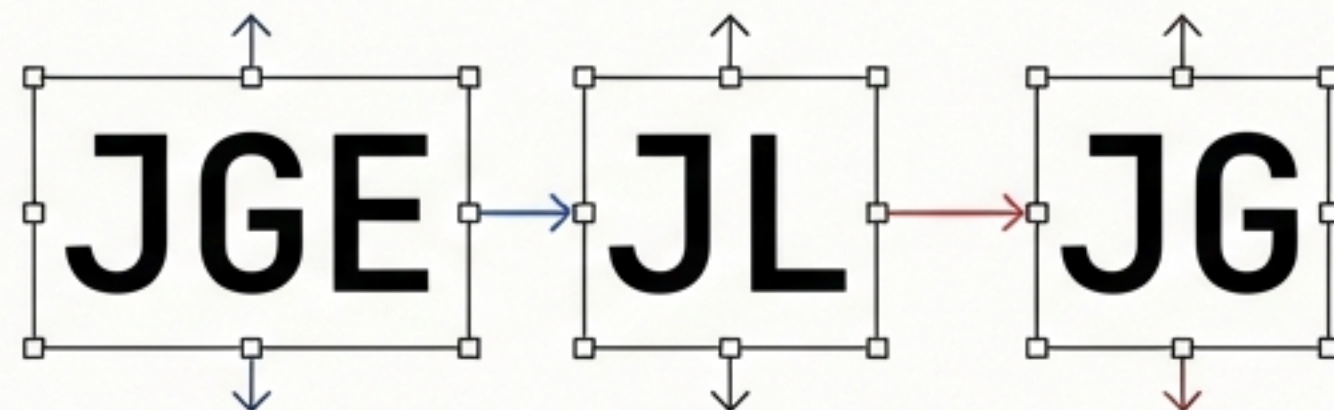
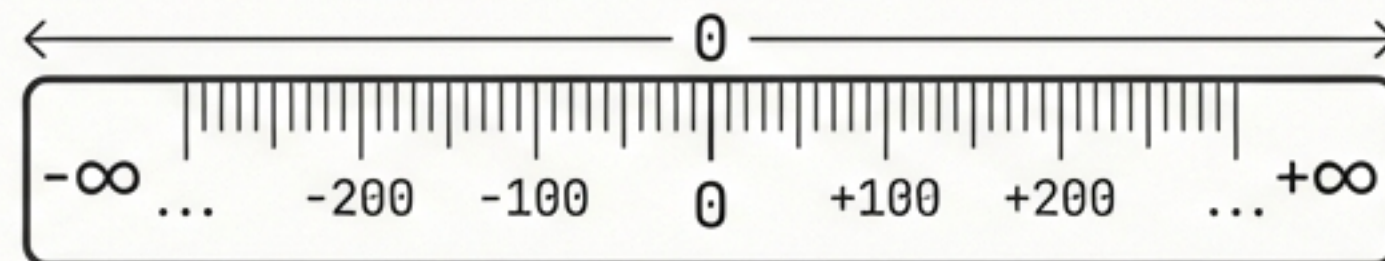
The CPU sees bits. The instruction reveals the intent.

Unsigned Integers



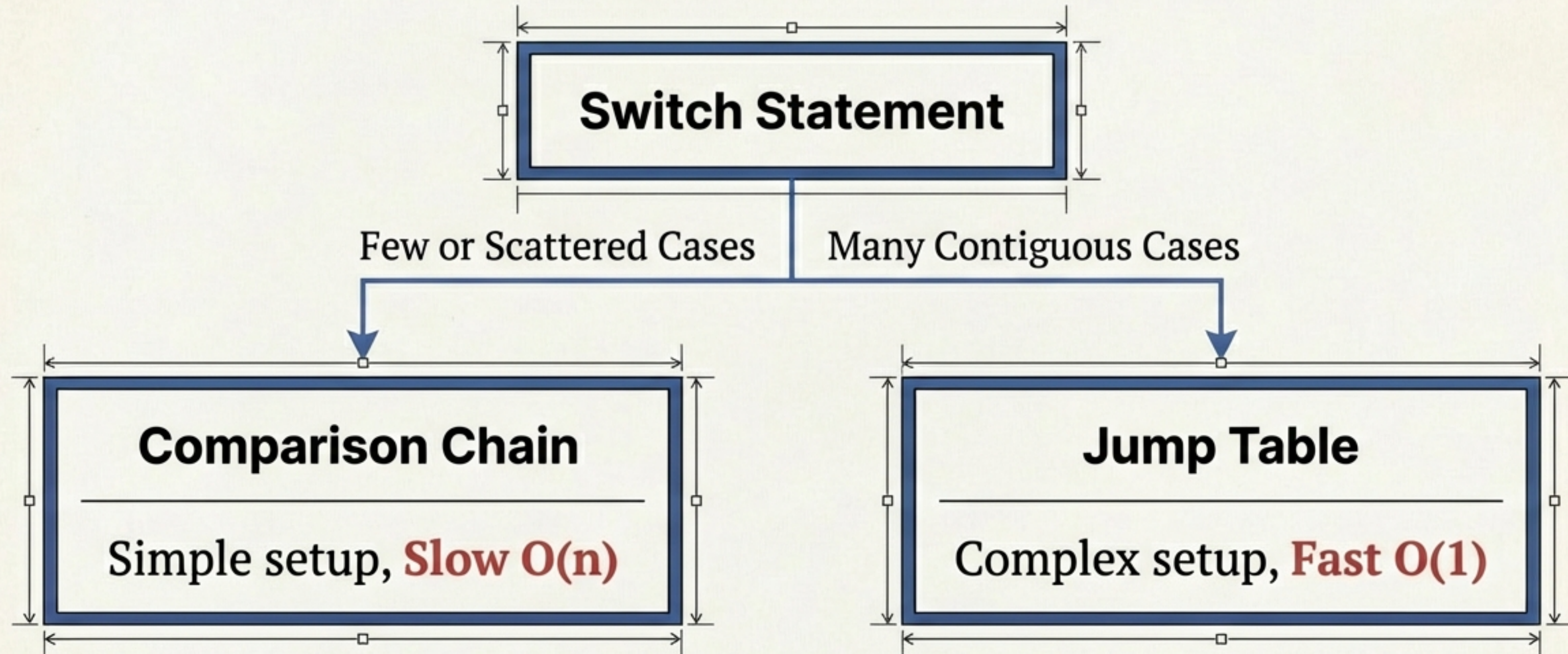
Used for memory addresses, sizes, or raw counters.

Signed Integers



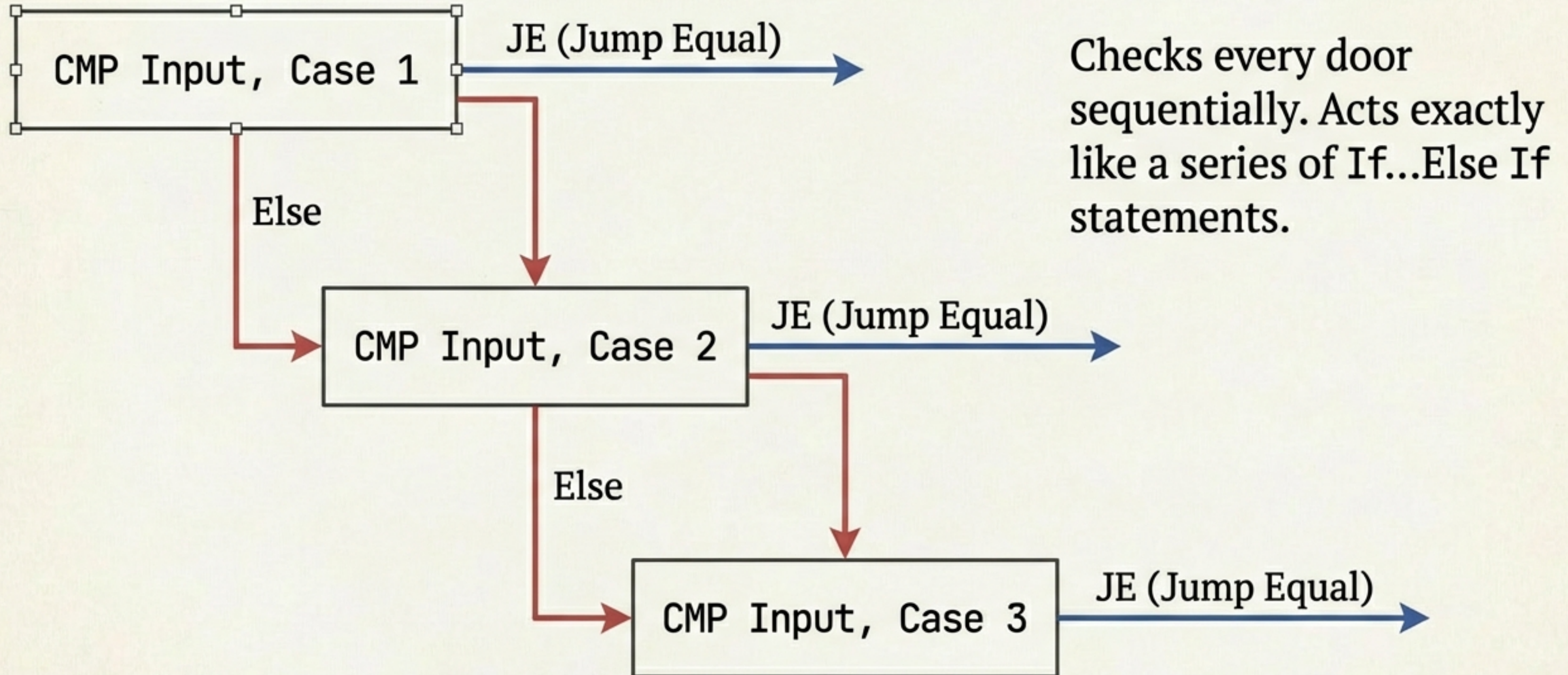
Used for arithmetic values (e.g., -10 to +10).

The Compiler's Decision

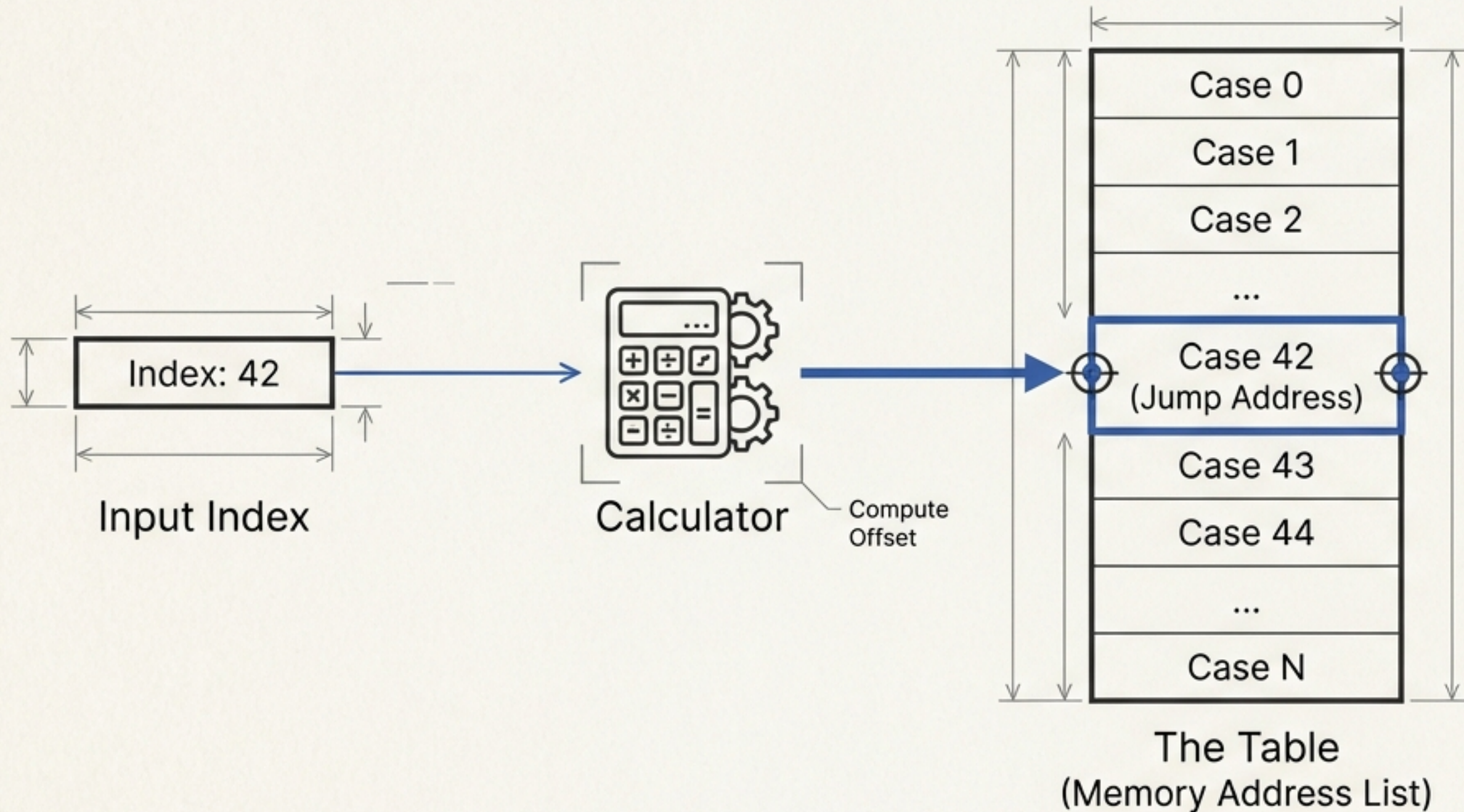


Threshold: Usually > 4 cases triggers a Jump Table.

Type A: The Comparison Chain



Type B: The Jump Table



✓ 1. Bounds Check ⚠
(Is index valid?)

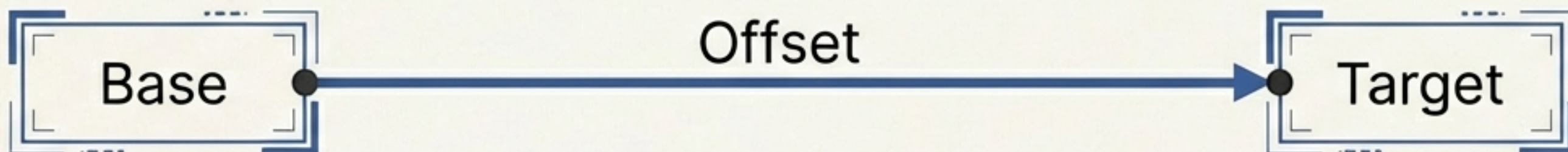
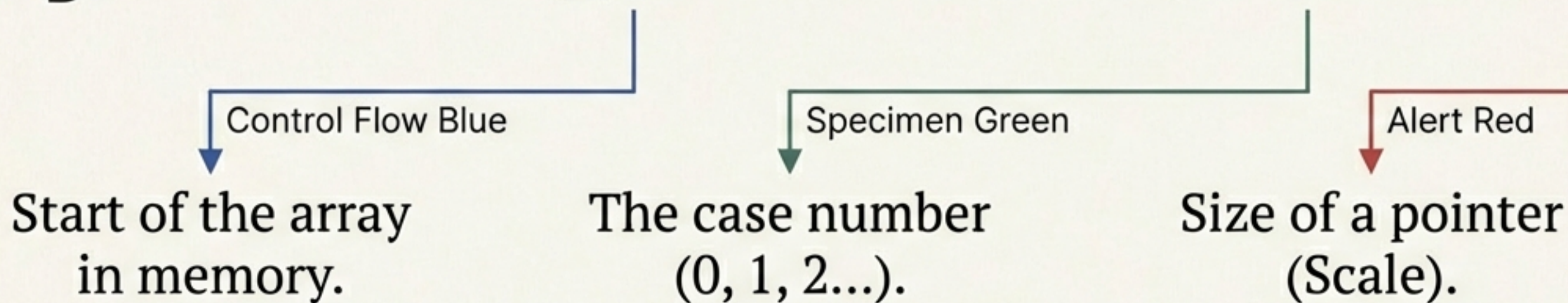
🔍 2. Access Table
(Get Address)

➡ 3. Direct Jump
(Go to Location)

$O(1)$ Constant Time.
Speed is independent of
the number of cases.

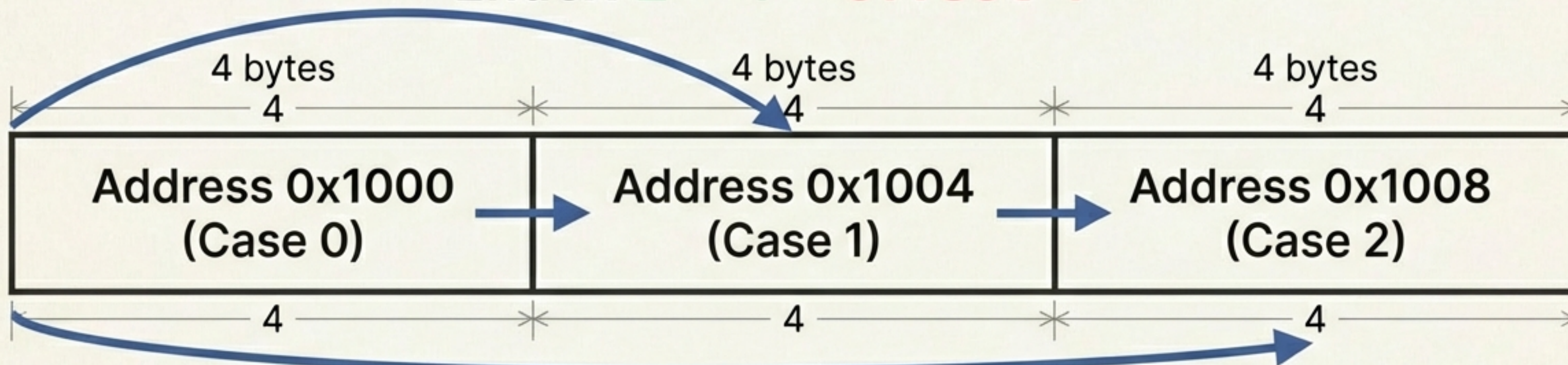
The Magic Formula

$$\text{Target} = \text{Base_Address} + (\text{Index} * 4)$$



Solving the 'Times 4' Mystery

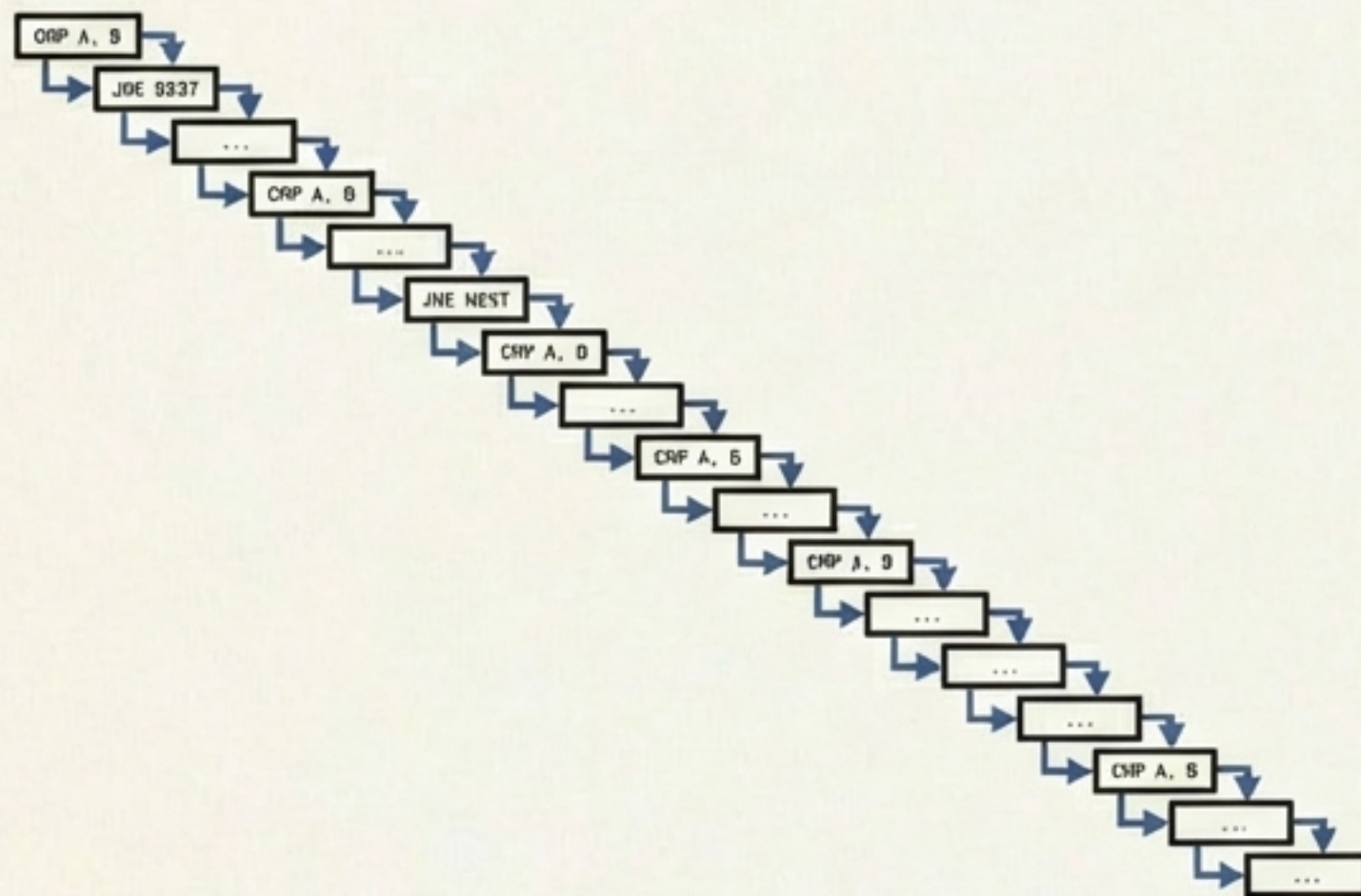
$$\text{Index } 1 \times 4 = \text{Offset } 4$$



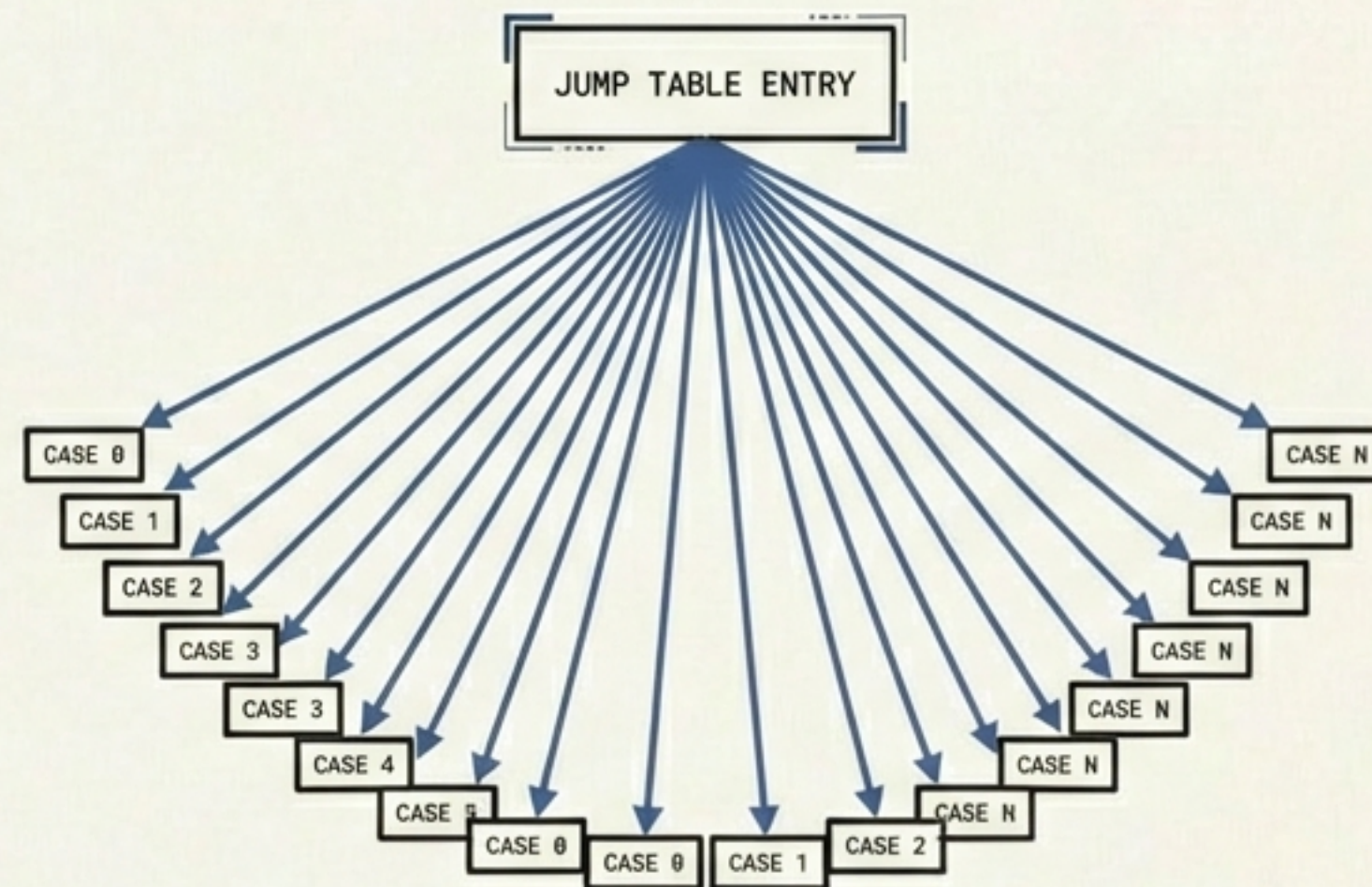
$$\text{Index } 2 \times 4 = \text{Offset } 8$$

In 32-bit architecture, every address is 4 bytes wide. To get from Case 0 to Case 1, you must skip 4 bytes. The multiplication converts a human index (1, 2) into a computer byte offset (4, 8).

The Shape of Efficiency



Comparison Chain (Slow)



Jump Table (Fast)

Recognize the shape in the graph view to instantly understand the code's complexity.

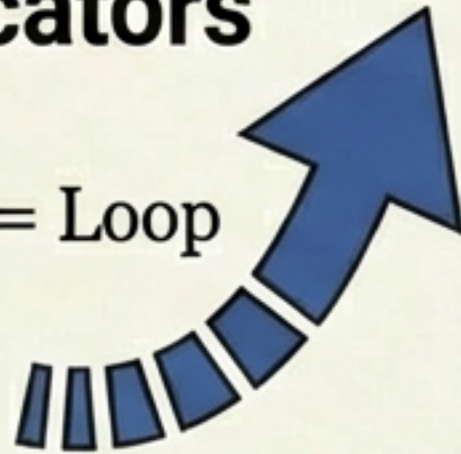
Knowledge Check

1. What is the primary visual indicator of a loop in IDA Pro?
A control flow arrow pointing backward (up).
2. If a loop uses 'JNB', is the variable Signed or Unsigned?
Unsigned.
3. Why is a Jump Table faster than a Chain?
It uses a mathematical lookup ($O(1)$) rather than checking every case sequentially.

Field Guide Cliffnotes

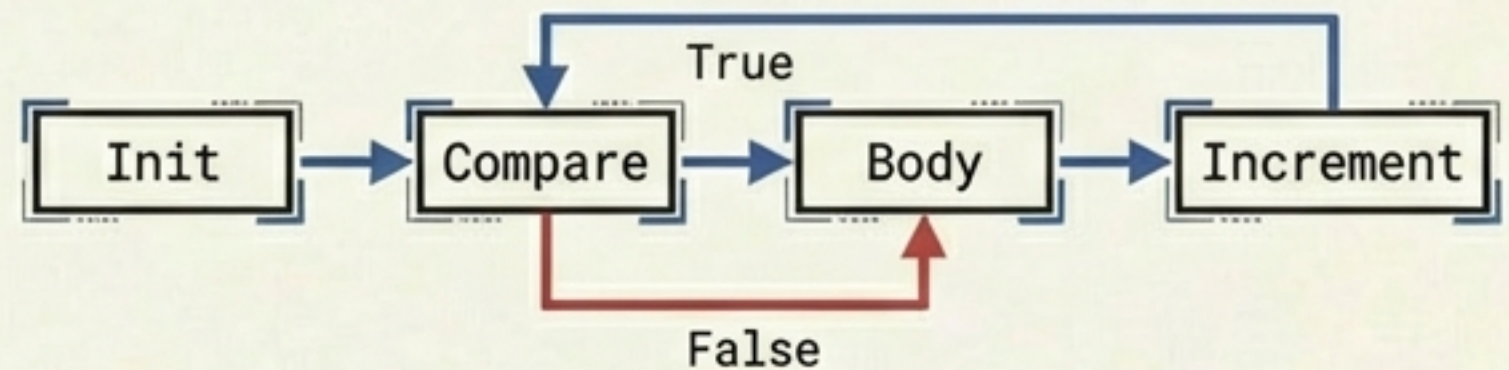
Visual Indicators

Blue Arrow Up = Loop

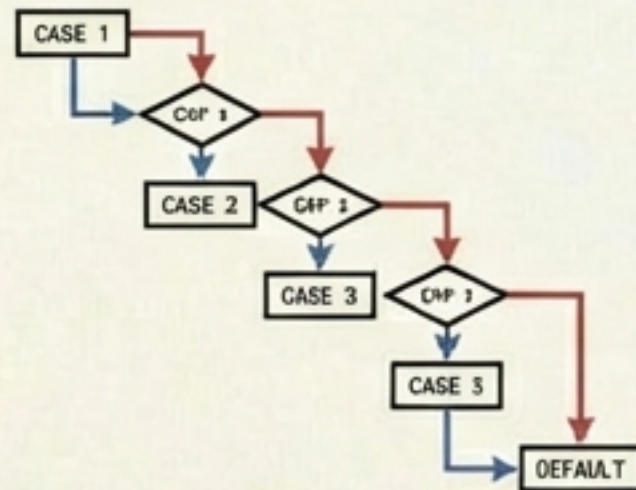


```
lost code = 8;
if ((Loop == 0) {
  car = [Jump Table, *4];
  det = 3;
  else {
    control(end ++, dod);
  }
}
```

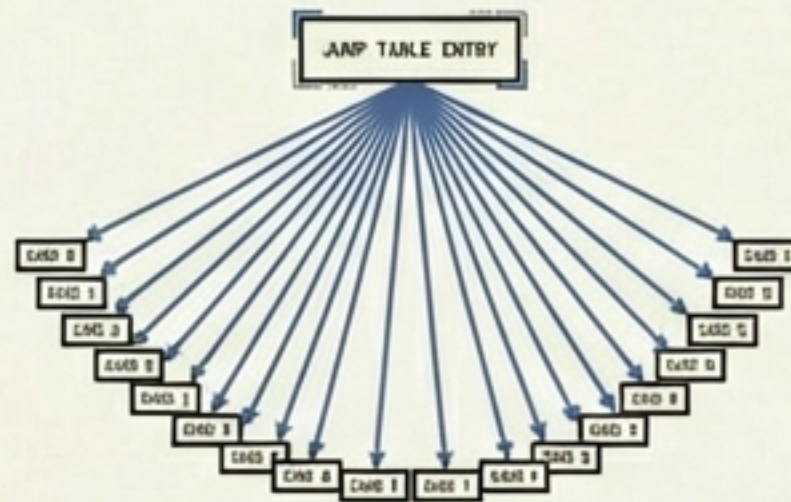
Loop Components



Switch Optimization



Few cases = Chain (If/Else)



Many cases = Table (Array Lookup)

The Math & Types

$[Register * 4] = \text{Jump Table} / \text{Array}.$

Register ×

--	--	--	--	--	--

 ...

--

 N

JGE/JL = Signed.

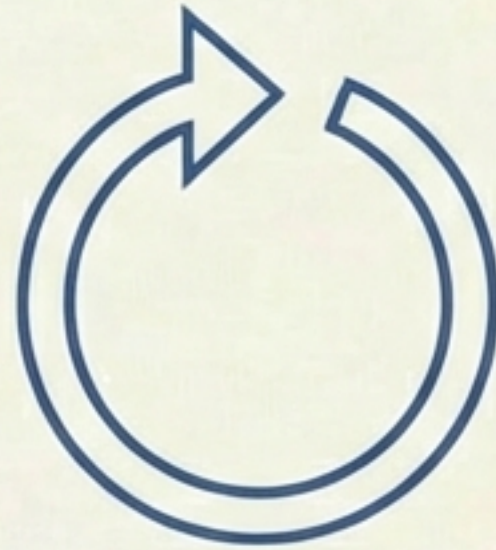
JA/JB = Unsigned.

Signed → +
→ -

Unsigned → +
→ -

Structure Over Syntax

Identifying these constructs allows you to ignore the noise. When you see the shape, you understand the logic without reading every line.



Course Lesson 07 Complete.